Monitor Financeiro BR

Documentação Técnica Completa

Análise de Sentimento de Notícias Financeiras
com Correlação a Ativos do Mercado Brasileiro

Autor: Ronald Andrade
Repositório: github.com/ronaldandrade/noticias
Produção: noticias-skp8.onrender.com

2025

Sumário

1 Visão Geral do Projeto **3**
1.1 Arquitetura em Camadas . . . 3

2 Stack Tecnológica e Dependências **3**
2.1 Dependências Principais . . . 3
2.2 Infraestrutura . . . 4

3 Modelo de Dados **4**
3.1 Tabela: `Noticia` . . . 4
3.2 Tabela: `Ativo` . . . 4
3.3 Tabela: `Cotacao` . . . 5
3.4 Tabela: `Correlacao` . . . 5

4 Pipeline de Execução **5**
4.1 Etapa 1 — Coleta de Notícias (Scraper) . . . 5
4.1.1 Fontes ativas . . . 6
4.1.2 Lógica de deduplicação . . . 6
4.1.3 Tick de coleta . . . 6
4.2 Etapa 2 — Análise de Sentimento . . . 6
4.3 Etapa 3 — Cotações . . . 6
4.4 Etapa 4 — Correlação . . . 7

5 Algoritmo de Análise de Sentimento **7**
5.1 VADER — Valence Aware Dictionary and sEntiment Reasoner . . . 7
5.2 Funcionamento Interno . . . 7
5.3 Léxico Financeiro Customizado . . . 8
5.4 Score Final Combinado . . . 8
5.5 Classificação do Sentimento . . . 8

6 Algoritmo de Associação Notícia → Ativo **9**
6.1 Problema e Motivação . . . 9
6.2 Sistema de Pontuação Ponderada . . . 9
6.3 Regras de Decisão . . . 9

7 Correlação Sentimento × Retorno **10**
7.1 Definição Formal . . . 10
7.2 Correlação de Pearson . . . 10
7.3 Correlação de Spearman . . . 10
7.4 Interpretação Prática . . . 10
7.5 Limitações e Considerações . . . 11

8 Interface Web **11**
8.1 Páginas Disponíveis . . . 11
8.2 API de Administração . . . 11

9 Pipeline CLI **12**

10 Automação em Produção **12**
10.1 GitHub Actions 12
10.2 Estratégia de Banco em Produção 12

11 Estrutura de Arquivos **13**

12 Próximos Passos e Evoluções Possíveis **13**
12.1 Curto Prazo 13
12.2 Médio Prazo 14
12.3 Longo Prazo 14

1. Visão Geral do Projeto

O **Monitor de Notícias Financeira** é uma aplicação que coleta notícias de fontes financeiras brasileiras, analisa automaticamente o sentimento de cada notícia e correlaciona esse sentimento com o retorno histórico de ativos do mercado financeiro: ações da B3, índices, câmbio e commodities.

O objetivo central é responder à pergunta: *notícias com sentimento negativo precedem quedas no preço de ativos?* A resposta é quantificada através de coeficientes de correlação calculados periodicamente sobre o histórico acumulado.

Premissa do projeto

O sentimento coletivo de notícias financeiras contém sinal preditivo sobre o comportamento de curto prazo dos mercados. Medir esse sinal de forma sistemática e contínua é o objetivo central desta aplicação.

1.1. Arquitetura em Camadas

A aplicação é organizada em cinco camadas independentes:

1. **Coleta** — Scraper HTML/RSS que varre múltiplas fontes financeiras (noticiário/cotação)
2. **Análise** — Serviço de sentimento VADER com léxico financeiro customizado
3. **Associação** — Algoritmo de pontuação ponderada para vincular notícias a ativos
4. **Correlação** — Cálculo estatístico de Pearson e Spearman entre sentimento e retorno
5. **Visualização** — Interface web Flask com dashboard e relatório interativo (Plotly)

2. Stack Tecnológica e Dependências

2.1. Dependências Principais

Biblioteca	Versão	Função no projeto
Flask	3.1.x	Framework web — rotas, templates, blueprints
Flask-SQLAlchemy	3.1.x	ORM — mapeamento objeto-relacional
Flask-Migrate	4.1.x	Migrations de banco de dados via Alembic
SQLAlchemy	2.0.x	Engine SQL e gerenciamento de sessões
vaderSentiment	3.3.2	Análise de sentimento baseada em léxico
yfinance	1.2.x	Download de cotações históricas via Yahoo Finance
scipy	1.17.x	Cálculo de correlação de Pearson e Spearman

Biblioteca	Versão	Função no projeto
pandas	3.0.x	Manipulação de séries temporais e alinhamento de dados
numpy	2.4.x	Operações numéricas vetorizadas
BeautifulSoup4	4.14.x	Parsing HTML/XML para raspagem de notícias
requests	2.32.x	Requisições HTTP para scraping
nltk	3.9.x	Tokenização e análise de trigramas no dashboard
scikit-learn	1.4.x	Utilitários de ML (disponível, não central nesta versão)
gunicorn	—	Servidor WSGI para produção no Render

2.2. Infraestrutura

- **Banco de dados:** SQLite em desenvolvimento, escalável para PostgreSQL
- **Hospedagem:** Render.com (plano gratuito com sleep automático)
- **Agendamento:** GitHub Actions (gratuito) — chama endpoint a cada 4h
- **Controle de versão:** Git/GitHub — deploy automático via push

3. Modelo de Dados

O banco de dados é composto por quatro tabelas principais com relacionamentos bem definidos.

3.1. Tabela: `Noticia`

Armazena cada notícia coletada com seu score de sentimento e vínculo com o ativo correspondente.

Campo	Tipo	Descrição
id	INTEGER PK	Identificador único
titulo	VARCHAR	Título da notícia
conteudo	TEXT	Corpo/descrição da notícia
url	VARCHAR UNIQUE	URL original — previne duplicatas
fonte	VARCHAR	Nome da fonte (InfoMoney, G1...)
resumo	TEXT	Resumo gerado automaticamente
score_sentimento	FLOAT	Score entre -1.0 e $+1.0$
ativo_id	INTEGER FK	Referência ao ativo associado
data_publicacao	DATETIME	Data/hora de publicação

3.2. Tabela: `Ativo`

Cadastro dos ativos financeiros monitorados.

Campo	Tipo	Descrição
id	INTEGER PK	Identificador único
ticker	VARCHAR UNIQUE	Código yfinance (ex: PETR4.SA, ^BVSP)
nome	VARCHAR	Nome de exibição
setor	VARCHAR	Setor econômico

3.3. Tabela: `Cotacao`

Preços históricos diários por ativo, obtidos via yfinance.

Campo	Tipo	Descrição
id	INTEGER PK	Identificador único
ativo_id	INTEGER FK	Referência ao ativo
data	DATE	Data do pregão
preco_fechamento	FLOAT	Preço de fechamento
preco_abertura	FLOAT	Preço de abertura
variacao_pct	FLOAT	Retorno diário em %
volume	BIGINT	Volume negociado

A restrição `UNIQUE(ativo_id, data)` garante que não haja duplicatas por ativo e data.

3.4. Tabela: `Correlacao`

Armazena os resultados calculados de correlação para cada período e ativo.

Campo	Tipo	Descrição
id	INTEGER PK	Identificador único
ativo_id	INTEGER FK	Referência ao ativo
data_inicio	DATE	Início do período analisado
data_fim	DATE	Fim do período analisado
pearson	FLOAT	Coeficiente de Pearson
spearman	FLOAT	Coeficiente de Spearman
n_noticias	INTEGER	Número de notícias no período
criado_em	DATETIME	Timestamp do cálculo

4. Pipeline de Execução

O pipeline completo percorre quatro etapas em sequência. Pode ser disparado manualmente via CLI, automaticamente pelo GitHub Actions, ou via endpoint HTTP.

4.1. Etapa 1 — Coleta de Notícias (Scraper)

O scraper suporta dois modos de coleta: raspagem de HTML direto e leitura de feeds RSS (como fallback para fontes que ainda os suportam).

4.1.1. Fontes ativas

- **InfoMoney** — páginas de ativo (`/tudo-sobre/petrobras/`) e mercados
- **G1 Economia** — seção de economia e feed RSS
- **Exame** — economia, mercados e commodities
- **UOL Economia** — feed RSS e página de seção
- **Suno Research** — notícias gerais e páginas por ativo
- **Livecoins** — feed RSS para criptomoedas

4.1.2. Lógica de deduplicação

Cada notícia é identificada unicamente pela URL. O sistema verifica duplicatas em dois níveis: em memória durante a execução (evita salvar duas notícias da mesma URL na mesma rodada) e contra o banco de dados (evita reinserir notícias já existentes).

4.1.3. Tick de coleta

```python
def buscar_noticias() -> list[Noticia]:
    ativos = Ativo.query.all()
    todas_raw = []

    for fonte in FONTES_HTML:
        if fonte["seletor_item"] is None:
            itens = _raspar_rss(fonte, ativos)
        else:
            itens = _raspar_html(fonte, ativos)
        todas_raw.extend([(item, fonte["nome"]) for item in itens])
        time.sleep(1.5)  # delay para evitar rate limit

    novas = [
        _montar_noticia(item, nome, ativos, num_frases=2)
        for item, nome in todas_raw
        if item["link"] not in urls_vistas
        and not Noticia.query.filter_by(url=item["link"]).first()
    ]

    db.session.bulk_save_objects(novas)
    db.session.commit()
    return novas
```

Listing 1: Fluxo principal do scraper

4.2. Etapa 2 — Análise de Sentimento

O score de sentimento é calculado diretamente na coleta, sem etapa separada.

4.3. Etapa 3 — Cotações

Cotações são baixadas via `yfinance` para todos os ativos cadastrados, com janela configurável (padrão: 90 dias). A variação percentual diária é calculada e armazenada:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}} \times 100 \tag{1}$$

onde P_t é o preço de fechamento no dia t.

4.4. Etapa 4 — Correlação

As notícias e cotações são alinhadas por data e a correlação é calculada entre o score médio diário de sentimento e a variação percentual do ativo.

5. Algoritmo de Análise de Sentimento

5.1. VADER — Valence Aware Dictionary and sEntiment Reasoner

VADER é um modelo de análise de sentimento baseado em léxico e regras, desenvolvido por Hutto & Gilbert (2014) especificamente para textos curtos e linguagem informal. Ao contrário de modelos de aprendizado de máquina supervisionado, VADER não requer treinamento: o conhecimento está codificado em um dicionário de palavras com pesos de valência.

Por que VADER e não um modelo de deep learning?

VADER foi escolhido por três razões práticas: (1) roda 100% localmente sem GPU ou API externa, processando centenas de notícias por segundo; (2) não requer corpus anotado em português para treinamento; (3) permite extensão trivial com léxico customizado do domínio financeiro.

5.2. Funcionamento Interno

Para cada texto, VADER calcula quatro escores:

- *pos* — proporção de tokens positivos
- *neg* — proporção de tokens negativos
- *neu* — proporção de tokens neutros
- *compound* — escore composto normalizado entre -1.0 e $+1.0$

O escore *compound* é calculado pela soma das valências de cada token, ajustada por regras contextuais (negação, intensificadores, pontuação), e normalizada pela função:

Normalização do escore composto

$$\text{compound} = \frac{x}{\sqrt{x^2 + \alpha}} \tag{2}$$

onde x é a soma bruta das valências e $\alpha = 15$ é um fator de normalização.

5.3. Léxico Financeiro Customizado

O léxico padrão do VADER é orientado a inglês e redes sociais. Para o domínio financeiro em português, foi construído um léxico customizado com mais de 60 termos e pesos calibrados manualmente:

```
LEXICO_FINANCEIRO = {
    # Positivos PT      pesos entre +1.0 e +4.0
    "lucro": 2.5,        "recorde": 2.5,
    "dividendo": 1.5,    "valorizacao": 2.2,
    "superou": 2.0,      "expansao": 1.8,

    # Negativos PT
    "prejuizo": -2.5,    "falencia": -3.5,
    "calote": -3.0,      "rebaixamento": -2.5,
    "fraude": -3.0,      "crise": -2.5,

    # Positivos EN
    "profit": 2.5,       "bullish": 2.2,
    "rally": 2.0,        "beat": 1.8,

    # Negativos EN
    "bankruptcy": -3.5, "bearish": -2.2,
    "plunge": -2.5,      "downgrade": -2.3,
}

_analyzer.lexicon.update(LEXICO_FINANCEIRO)
```

Listing 2: Trecho do léxico financeiro PT+EN

5.4. Score Final Combinado

O score final combina o VADER com uma média ponderada do léxico financeiro:

Fórmula do score combinado

$$S_{\text{final}} = 0.6 \cdot S_{\text{VADER}} + 0.4 \cdot S_{\text{léxico}} \tag{3}$$

onde $S_{\text{léxico}}$ é calculado como:

$$S_{\text{léxico}} = \text{clip}\left(\frac{\sum_{w \in T} \text{peso}(w)}{|T| \cdot 4},\ -1,\ +1\right) \tag{4}$$

sendo T o conjunto de tokens presentes no léxico financeiro.

5.5. Classificação do Sentimento

Intervalo	Classe	Interpretação
$S > 0.05$	Positivo	Tom otimista, boas notícias
$-0.05 \leq S \leq 0.05$	Neutro	Tom informativo sem carga emocional
$S < -0.05$	Negativo	Tom pessimista, más notícias

6. Algoritmo de Associação Notícia → Ativo

6.1. Problema e Motivação

O problema central da associação é que notícias financeiras frequentemente mencionam múltiplos ativos ou usam termos genéricos. Uma notícia sobre "a bolsa recuou com queda do petróleo" menciona tanto o Ibovespa quanto a Petrobras. O algoritmo precisa decidir qual ativo é o *tema principal*.

A versão inicial usava um critério de primeira ocorrência: o primeiro ativo cujas palavras-chave aparecessem no texto era associado. Isso causava erros sistemáticos — notícias sobre dólar eram associadas ao Ibovespa porque "B3" ou "bolsa" apareciam no contexto.

6.2. Sistema de Pontuação Ponderada

O algoritmo atual atribui uma pontuação a cada ativo com base nas palavras-chave encontradas no texto. Palavras mais específicas recebem pesos maiores:

Pontuação por ativo

$$P(a) = \sum_{w \in \text{título}} 2 \cdot \text{peso}(w, a) + \sum_{w \in \text{conteúdo}} \text{peso}(w, a) \tag{5}$$

O título tem peso dobrado porque uma menção no título indica tema principal.

```
KEYWORDS_PONDERADAS = {
    "PETR4.SA": {
        "PETR4":      10,  # ticker expl cito     match quase certo
        "PETROBRAS":   5,  # nome pr prio da empresa
        "PRE-SAL":     3,  # termo espec fico do setor
        "PETROLEO":    2,  # termo setorial (pode ser contexto)
        "GASOLINA":    1,  # gen rico      s   refor a
    },
    "^BVSP": {
        "IBOVESPA":    5,
        "BOVESPA":     3,
        "B3":          2,  # reduzido     muito gen rico
    },
}
```

Listing 3: Hierarquia de pesos no léxico de associação

6.3. Regras de Decisão

1. Calcula $P(a)$ para todos os ativos cadastrados
2. O ativo com maior $P(a)$ é o candidato
3. Se $P(a) < 4$ (limiar mínimo), a notícia fica **sem ativo** — sem associação forçada
4. Se o feed tem um `ticker_hint` (ex: feed do InfoMoney PETR4), esse ticker tem prioridade absoluta

Princípio fundamental

É melhor não associar uma notícia a nenhum ativo do que associá-la ao ativo errado. Associações incorretas contaminam o cálculo de correlação com ruído.

7. Correlação Sentimento × Retorno

7.1. Definição Formal

Dado um ativo a e uma janela temporal $[t_0, t_1]$, define-se:

- $\bar{S}_t$ — score médio de sentimento das notícias associadas ao ativo a publicadas no dia t
- r_t — retorno percentual do ativo a no dia t

A correlação é calculada sobre os dias em que *ambos* os valores estão disponíveis (interseção por data).

7.2. Correlação de Pearson

Mede a relação linear entre as duas séries:

Coeficiente de Pearson

$$r = \frac{\sum_{t=1}^{n}(\bar{S}_t - \bar{S})(r_t - \bar{r})}{\sqrt{\sum_{t=1}^{n}(\bar{S}_t - \bar{S})^2 \cdot \sum_{t=1}^{n}(r_t - \bar{r})^2}} \tag{6}$$

$r \in [-1, +1]$. Valores próximos de $+1$ indicam que sentimento positivo se associa a retornos positivos. Valores próximos de -1 indicam relação inversa.

7.3. Correlação de Spearman

Mede a relação monotônica (não necessariamente linear), sendo mais robusta a outliers:

Coeficiente de Spearman

$$\rho = 1 - \frac{6\sum_{t=1}^{n} d_t^2}{n(n^2-1)} \tag{7}$$

onde $d_t = \text{rank}(\bar{S}_t) - \text{rank}(r_t)$.

7.4. Interpretação Prática

$\lvert r\rvert$ ou $\lvert\rho\rvert$	Força	Interpretação
≥ 0.5	Forte	Sinal consistente e relevante
$0.3 - 0.5$	Moderada	Tendência presente, com ruído
< 0.3	Fraca	Sinal fraco ou dados insuficientes

Por que usar os dois coeficientes?

Pearson assume que a relação é linear e é sensível a outliers extremos (um crash ou uma alta atípica pode distorcer o resultado). Spearman é baseado em rankings e não faz essa suposição. Quando ambos convergem para valores similares, o resultado é mais confiável.

7.5. Limitações e Considerações

- **Causalidade vs. correlação:** correlação não implica causalidade. O sentimento pode ser uma consequência dos preços, não a causa.
- **Defasagem temporal:** o modelo atual alinha notícias e retornos pelo *mesmo dia*. Modelos mais sofisticados testariam defasagens de 1, 2 ou 3 dias.
- **Volume mínimo:** são necessários pelo menos 5 pontos de interseção para calcular correlação. Com poucos dados, o coeficiente é estatisticamente irrelevante.
- **Ruído da associação:** notícias mal associadas introduzem ruído e atenuam o sinal verdadeiro.

8. Interface Web

8.1. Páginas Disponíveis

Rota	Descrição
`/`	Lista paginada de notícias com badge de sentimento, ticker do ativo, fonte e data. Cards de visão geral no topo (totais, score médio, positivas/negativas).
`/dashboard`	Trigramas mais frequentes nos títulos das notícias. Análise de frequência de assuntos por período.
`/relatorio`	Gráficos interativos (Plotly): série temporal sentimento × preço, heatmap sentimento por dia da semana, tabela de correlações, últimas notícias analisadas. Filtrável por período (30/60/90/180 dias).
`/noticia/<id>`	Detalhe completo de uma notícia individual.

8.2. API de Administração

Método	Rota	Função
GET	`/admin/status`	Contagens do banco (health check)
GET	`/admin/scores`	Últimas 50 notícias com score (debug)
POST	`/admin/scoring`	Pipeline completo via HTTP
POST	`/admin/atualizar-completo`	Scraper + scoring + correlações
POST	`/admin/cron/atualizar`	Endpoint do GitHub Actions (com auth)

9. Pipeline CLI

Após o refactor, todos os comandos de manutenção foram unificados no grupo `flask pipeline`:

```
# Ver situa  o do banco dados
flask pipeline status

# Coletar novas noticias
flask pipeline scraper

# Calcular scores e correlacoes
flask pipeline scoring --dias 90

# Reatribuir noticias aos ativos (algoritmo novo)
flask pipeline reatribuir

# Pipeline completo em sequencia
flask pipeline tudo --dias 90

# Zerar todos os scores (requer confirmacao)
flask pipeline resetar-scores
```

Listing 4: Comandos CLI disponíveis

10. Automação em Produção

10.1. GitHub Actions

O scraper é executado automaticamente três vezes ao dia via GitHub Actions. O workflow chama o endpoint `/admin/cron/atualizar` com chave secreta de autenticação:

```
on:
  schedule:
    - cron: '0 9,15,21 * * *'  # 9h, 15h, 21h UTC
  workflow_dispatch:             # permite disparo manual

jobs:
  scraper:
    runs-on: ubuntu-latest
    steps:
      - name: Chamar endpoint de atualizacao
        run: |
          curl -s -X POST ${{ secrets.SITE_URL }}/admin/cron/atualizar \
            -H "X-Cron-Secret: ${{ secrets.CRON_SECRET }}" \
            --retry 3 --retry-delay 15
```

Listing 5: Workflow de agendamento (.github/workflows/scraper.yml)

10.2. Estratégia de Banco em Produção

Desenvolvimento e produção mantêm bancos independentes por design — o arquivo `.db` está no `.gitignore`. O banco de produção acumula dados organicamente através do

agendamento automático. Para sincronizar dados históricos, o endpoint `/admin/scoring` pode ser chamado manualmente com uma janela maior de dias.

11. Estrutura de Arquivos

```
noticias/
  run.py                      # Entry point Flask
  test.py                     # Diagnostico completo do pipeline
  requirements.txt            # Dependencias de producao
  .github/
    workflows/
      scraper.yml             # GitHub Actions     agendamento
  app/
    __init__.py               # Factory da aplicacao, registro de
        blueprints
    models.py                 # Noticia, Ativo, Cotacao, Correlacao
    routes.py                 # Blueprints: main, admin, relatorio
    scraper.py                # Coleta HTML + RSS
    repository.py             # Queries paginadas e filtradas
    cli.py                    # Comandos flask pipeline
    services/
      sentimento_service.py   # VADER + lexico financeiro
      associacao_service.py   # Algoritmo de pontuacao ponderada
      cotacao_service.py      # yfinance + calculo de correlacao
      relatorio_service.py    # Dados para o template de relatorio
      resume_text_service.py # Geracao de resumos
    templates/
      index.html              # Pagina principal
      dashboard.html          # Dashboard de trigramas
      relatorio.html          # Relatorio com graficos Plotly
      noticia.html            # Detalhe de noticia
    static/
      styles.css              # Estilos globais
```

Listing 6: Estrutura do projeto

12. Próximos Passos e Evoluções Possíveis

12.1. Curto Prazo

- Adicionar defasagem temporal na correlação (sentimento do dia t vs. retorno do dia $t+1$)
- Alertas automáticos por e-mail/Telegram quando o score médio cair abaixo de um limiar
- Migrar banco de dados para PostgreSQL (Supabase) para persistência mais robusta em produção

12.2. Médio Prazo

- Substituir VADER por um modelo transformer fine-tuned para português financeiro (ex: BERTimbau ou FinBERT-PT)
- Implementar análise de defasagem cruzada (cross-correlation) com lag variável de 0 a 5 dias
- Adicionar análise de entidades nomeadas (NER) para identificação automática de empresas no texto

12.3. Longo Prazo

- Modelo preditivo de retorno com features de sentimento, volume de notícias e indicadores técnicos
- API pública para consumo dos scores por terceiros
- Expansão para mercados internacionais (S&P 500, notícias em inglês)

Referências

[1] Hutto, C.J. & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. *ICWSM-14*.

[2] Spearman, C. (1904). The proof and measurement of association between two things. *American Journal of Psychology*, 15, 72–101.

[3] Pearson, K. (1895). Notes on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242.

[4] yfinance — Yahoo Finance market data downloader. https://github.com/ranaroussi/yfinance

[5] Flask — The Pallets Projects. https://flask.palletsprojects.com

[6] SQLAlchemy — The Python SQL Toolkit and ORM. https://www.sqlalchemy.org